MDPI

Article

HARK: Human-Gated Authority-Typed Review Knowledge for LLM-Assisted Proposal Review

Author A [1], Author B [1], Author C [1], Author D [1], Author E [1], Author F [1]

[1] Digital Technology Department, Example Utility Company, Beijing 100000, China; author.a@example.com (A.); author.b@example.com (B.); author.c@example.com (C.); author.d@example.com (D.); author.e@example.com (E.); author.f@example.com (F.)

* Correspondence: author.a@example.com

Abstract

Large language models (LLMs) increasingly pre-screen technical documents before an expert signs off, yet expert review knowledge enters such systems as keyword rules, prompt checklists or retrieval over past reviews, and each route loses a property that high-stakes review needs: findings must trace to a rule and a document location, decision authority must match how deterministically a requirement can be checked, knowledge must grow from use, and that growth must stay under human control. We call this the *review-knowledge ingestion problem* and study it for preliminary review of technical solution proposals at a large utility. We present HARK (Human-Gated Authority-Typed Review Knowledge), which answers the four properties with four mechanisms rather than one. Knowledge becomes a versioned package of verifiability-typed rules: *hard* rules are owned by deterministic verifiers that produce located evidence, including absence evidence an LLM may never assert; *hybrid* rules let verifiers locate candidate spans while the LLM judges only inside them, enforced by a tool-side evidence gate; *soft* rules only raise items for judgement. A deterministic policy derives the suggested verdict; only the expert writes the final one. Expert decisions become an audited learned layer that may tune parameters and phrasing but never promotes a rule's tier. Implemented as a plugin of an open-source agent harness (28 rules, no separate backend), the design reaches F1 0.798 against 0.222 for a keyword baseline on 100 synthetic proposals with 253 planted defects, with no false positive and 85% verdict agreement ($\kappa = 0.76$); 80% of the remaining misses reach the expert as open items. Only 10 of 28 rules proved deterministically verifiable, and the four blocking rules least of all.

Keywords: large language models; automated document review; human-in-the-loop; neuro-symbolic verification; evidence grounding; verifiability typing; expert feedback; knowledge evolution

Received:
Revised:
Accepted:
Published:

Copyright: © 2026 by the authors. Submitted to *Computers* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Before a change to a large online service platform is built, its *technical solution proposal*—a templated document describing background, architecture, interfaces, data flows, implementation plan and risk—must be reviewed by a small group of senior experts. The review is checklist-driven, the documents are highly templated, the consequences are real (the expert signs the opinion), and the knowledge is both specialised and changing. These properties make the preliminary review a natural target for LLM assistance; they also make the usual way of building that assistance fragile.

Our starting point is a failed first attempt. A monolithic prototype built in 2025 encoded 24 expert rules as a few keywords each and delegated everything else to an LLM through long prompts. "Load test" appearing anywhere counted as the load test having been planned; negations ("no load test is planned for now"), placeholders ("/", "N/A") and empty sections were invisible; the verdict was whatever the model wrote into a single field, so a blocking finding did not change the outcome; and nothing was persisted, so the hundredth review was no better informed than the first. A faithful reimplementation of that design, evaluated below on the same corpus as the design we propose, recovers 18.6% of planted defects, produces 2.6 false findings for every true one, and agrees with the expected verdict no better than chance ($\kappa = -0.04$). None of this was a failure of the model. All of it was a failure of *how the review knowledge had been put into the system*.

Take a single rule, "a load test must be planned before go-live". In a corpus of real proposals written for this platform, the sentence that satisfies it reads: "Load testing is completed by the test team one week before go-live and a report is issued." Three other formulations appear just as often. The negated one: "No stress testing is planned for the time being; it will be decided after observing traffic after go-live." The hollow one: "Load testing plan: to be determined", or "Load testing: /". And the satisfied-by-analogy one, in which the required content exists but the keyword family does not: "Before go-live we run the capacity drill in the pre-production environment; at 500 TPS the average response time must stay below 200 ms." All four contain the keyword that a rule-as-substrings implementation keys on. Only one of them is a pass, and the difference between them is not a difference in wording but in what part of the document they occupy and what a reader does with them.

Three ingestion routes dominate practice, and each loses a different property. *Keywording* turns a rule into substrings and loses its meaning: presence is not satisfaction and absence is not violation. *Prompting* turns a rule into instructions and loses auditability: the model has seen the rule, but its judgement cannot be traced to the rule or to a document location. *Fragmenting*—retrieval over past reviews—loses accumulation: knowledge is reassembled at every query and never consolidated. All three treat knowledge as input shown to the model rather than as an asset owned by the system, and all three make the same implicit assumption, that a requirement can be handed to a single mechanism.

Two tensions underlie that assumption. The *verifiability tension*: requirements span a spectrum from "the implementation-plan section exists and is not a placeholder", which a program can check exactly, to "the risk analysis is adequate", which is almost entirely judgement. The less verifiable a requirement, the more it needs a model; the more verifiable it is, the less a model should be trusted with it, because models make unexplained errors on deterministic questions and cannot locate them [1,2]. A representation that assigns every rule to one mechanism must therefore be wrong for most of the rule set, and the error is not random: it concentrates on exactly the rules an organisation cares about most. The *growth tension*: knowledge must grow from use—every dismissal is information—but where a human signs the outcome, growth cannot be unilateral, and growth in weights or filter thresholds is invisible to those accountable for it.

We present HARK, an open-source implementation of this representation as a plugin of an open agent harness, and evaluate it on a synthetic corpus of 100 proposals and on five internal proposals reviewed end to end. We make four contributions:

(1) We formulate the *review-knowledge ingestion problem* and derive four design requirements for a solution—*auditability*, *authority follows verifiability*, *growth*, *bounded growth*—together with the mechanism that enforces each requirement and the instrument that measures it (Table 2).

https://doi.org/10.3390/computers1010000

(2) A *verifiability-typed* rule model in which the type decides who may raise a finding: hard rules are owned by deterministic verifiers, hybrid rules admit the model only inside verifier-located spans, and soft rules only queue items for the expert. Absence is a verifier-owned evidence kind, candidates are never a pass, and a deterministic policy computes the suggested verdict from confirmed findings alone (Section 3.2, Figure 2).

(3) *Review as feedback*: the expert's ordinary review decisions—accept, dismiss, rewrite, add—are the only source of change to a learned layer that the system may tune but never promote, with an effective knowledge version stamped on every review (Section 3.6).

(4) A plugin-native implementation with no separate backend, database or front-end build, evaluated on a released synthetic corpus with planted defects against a faithful reimplementation of the keyword prototype, including per-tier, per-severity, per-defect-kind and threshold analyses, an evidence-gate audit and an end-to-end walkthrough (Section 4).

The design is assessed one question per property: **RQ1** can every finding be traced to a rule and a location in the document? **RQ2** does typed authority outperform untyped authority, and does the type predict where the remaining judgement sits? **RQ3** does the knowledge measurably grow, and **RQ4** is all growth human-attributable and reversible? Section 4 answers RQ1 and RQ2 with measurements on the released corpus, reports what RQ3 and RQ4 would require, and states explicitly which of the four properties this study does not yet measure.

Figure 1 summarises the resulting architecture. A proposal enters as Markdown (Word and PDF are converted upstream); the parser produces a structured document with source offsets; verifiers own everything that can be decided deterministically and prepare candidate spans for everything that cannot; the model sees only those spans and may only propose findings; the evidence gate admits a finding only if its quotation can be located inside the span the verifier chose; the policy turns confirmed findings into a suggested verdict; and the only path back into knowledge is the expert's decisions, which are written to an audited learned layer.

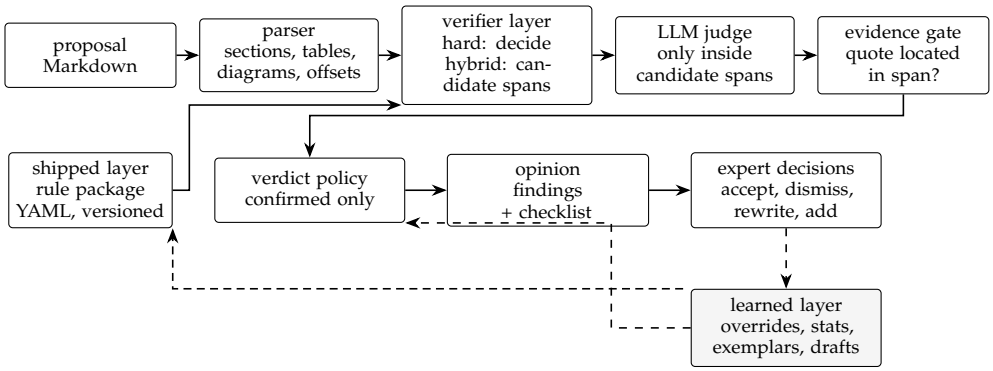


Figure 1. Architecture and the direction of authority. Solid arrows move a proposal through the review; dashed arrows are the only path by which knowledge changes. Hard checks are decided inside the verifier layer and never reach the model; the model may only propose a finding inside a span the verifier chose, and the gate rejects it if the quotation does not lie there. The learned layer may tune parameters, phrasing, severity and status of shipped rules and may draft new soft rules, but it cannot promote a rule's tier, so no dashed path increases what the system may decide alone.

2. Related Work

Inspection and requirements quality.

Checklist-based reading has been the backbone of document and code inspection since Fagan [3], and surveys stress that reading technique, not reviewer talent, determines yield [4]. Verifiability is a defined quality attribute of a requirement [5,6], and requirements smells—weak words, placeholders, missing units—are detected with lightweight deterministic analysers precisely because these are cheap, explainable and stable [7]. Recent work keeps the deterministic core and adds recommendation: Veizaga et al. detect smells in natural-language requirements and suggest how to remove them with a tool built on pre-trained language models [24], and AirReq applies smell detection and elimination to the requirements of commercial aircraft systems, where the cost of a missed ambiguity is measured in certification [25]. Our verifiers inherit this tradition, and like these systems we treat surface cues—negation, placeholders, missing units—as first-class evidence. The difference is scope and authority: smell analysers operate on individual requirement sentences and leave the judgement of a whole document to the reader, whereas our rules address a whole proposal and the type decides which of them may be decided automatically.

LLMs for requirements and design review.

Systematic reviews map how NLP and LLMs are used across requirements engineering [8], and surveys of hallucination supply the failure taxonomy we design against [9]; a dual-perspective review treats LLMs both as tools for software verification and as artifacts whose output must itself be verified [10]. Multi-agent validators such as MARV retrieve certification standards to check a requirement specification [11]; ReqPlan-Eval routes requirements to human review on evidence signals, keeps decision authority with the reviewer, and states what its evidence does not establish [12]. Outside requirements engineering the same question is studied as review automation: Pornprasit and Tantithamthavorn compare fine-tuning and prompt engineering for LLM-based code review and find that parameter-efficient fine-tuning dominates prompting at equal or lower cost [26], while an industrial reviewer is evaluated online for the quality and usefulness of the comments it produces [33]. Two background results frame our design. Model judges are not interchangeable across architectures: the same annotation task yields systematically different polarity for different model families [34], which is why we do not let a model's confidence stand in for a rule's authority. And evaluation of LLMs is itself an open methodological problem [29], which is why our corpus separates mechanism correctness from model capability instead of reporting one aggregate score.

Neuro-symbolic compliance review.

PolicyGuard compiles organisational policy into typed relational logic rules whose ground atoms are answered by an LLM from retrieved evidence before a symbolic evaluator applies the rule [13]. It satisfies auditability and partially separates authority, but the separation is *symmetric*—every atom is answered by the LLM—and it has no notion of a requirement the model must not be asked at all, no feedback loop and no bounded evolution. The same symmetric pattern recurs where symbolic constraints are used to catch abductive errors: Galitsky and Rybalov verify LLM diagnoses in process control against physical and sensor constraints, using the symbolic layer as a rejection filter rather than as an owner of individual checks [27]. Neuro-symbolic integration in general aims to let symbolic components constrain learned ones [14]; our contribution is a specific and static constraint—evidence must fall inside verifier-located spans, and a package that violates the pattern is refused at load time—plus a type that says, before any document is read, where the model will not be consulted.

Human-in-the-loop control.

The human-in-the-loop literature distinguishes active learning, in which the system keeps control of what it asks, interactive machine learning, in which control is shared, and machine teaching, in which the domain expert decides what is learned and when [28]. Reviewing a proposal is machine teaching whether or not anyone calls it that: the expert knows the rule, the document and the consequence. What such a protocol needs is not a new interaction metaphor but an accounting of authority—which decisions the system may take alone, which it may prepare, and which it must hand over. Our design fixes that accounting per rule, by type, and records it in the rule package rather than in a prompt.

Industrial review feedback and the limits of model judgement.

Industrial code review learns from developer reactions: BitsAI-CR runs a rule checker and then a learned filter that suppresses the false positives the rules raise [15], RovoDev reports an online evaluation of a deployed reviewer at Atlassian [33], and mining the feedback developers leave on an agentic reviewer in the wild shows that a large share of its comments is rejected, mostly as false positives or misalignment, and that rejection is predictable [16]. In all of these, learning happens in the model, a filter or a vendor-operated pruning step; the reviewer cannot see it or roll it back, and the object of review is code rather than a design document. A second limit concerns the input itself: a document under review is untrusted content, and an attacker who can write into it can address the judge directly—optimisation-based prompt injection against an LLM-as-a-judge succeeds without access to the judge's weights [32]. Any architecture in which the model is the only reader of the document inherits that attack surface, which is a further reason to have the verifier, not the model, decide what the model is allowed to read.

Grounding, cited evidence and memory.

Attributed generation attaches citations to claims [17], and a claim-level evaluation of verbatim quotation shows that producing a quote and substantiating a claim are not the same operation [18]. We add two constraints the citation literature does not need: quotes must lie in a span chosen by a verifier, and absence can never be claimed by the model. The same reasoning applies to what the system remembers. Agent memory is a productive research direction [30], but memories extracted from interactions can leak or be inverted to recover the content they summarise [31]; because our learned layer stores counts, enum-valued overrides, short quotations and exemplar sentences rather than free-form recollections, its exposure is bounded and its entries are individually auditable. Self-refinement and reflection let agents improve their own outputs or memory [19,20], and open-ended self-modifying agents exist [21]; we treat "which layer of the knowledge may the system change on its own" as a design decision fixed by type. Our protocol is machine teaching in the strict sense [28], and the dashboard exposes what has been learned per rule.

Positioning.

Table 1 places the design against the closest families on five properties: the form of the knowledge, who may decide a finding, whether evidence is located in the document, whether knowledge changes from use, and whether that change is bounded by something a reviewer can inspect. No prior system in the table combines per-rule typed authority with a located-evidence gate and a human-bounded learned layer; the closest, PolicyGuard, has the auditability but not the asymmetry, the feedback or the bound.

Table 1. Positioning. "Located" means a finding carries a document range or an explicit verifier-produced absence statement; "bounded" means a reviewer can inspect and reverse every knowledge change.

Approach	Knowledge form	Who decides	Located	Learns from use	Bounded
Keyword checklist (2025 prototype)	substrings per rule	program	span only, no semantics	no	—
Requirements-smell analysers [7,24,25]	deterministic cue rules (+ trained repair)	program	yes, sentence-level	no	—
MARV [11]	retrieved standards in role prompts	model	quotes standard, not location	no	—
ReqPlan-Eval [12]	evidence signals per requirement	reviewer (model routes)	evidence stated, limitations disclosed	not reported	—
PolicyGuard [13]	typed relational policy, LLM answers atoms	symbolic evaluator	yes, per atom	no	—
Industrial review agents [15,16,33]	prompts plus comment filters	model	partial	yes, in model or filter	not visible to reviewer
This work	typed rule package + audited learned layer	type decides per rule	yes: quote in verifier span, or absence (verifier only)	yes, from review decisions	yes: tier immutable, audited, reversible

3. Materials and Methods

3.1. Problem Formulation and Design Requirements

Let K be a body of review knowledge—rules, phrasing exemplars and past decisions—and S an LLM-based system that, given a document d, produces findings and a suggested verdict. The *review-knowledge ingestion problem* is to choose a representation of K and an assignment of decision authority over it such that:

(i) **Auditability.** Every finding traces to an element of K and to a location in d, or to an explicit verifier-produced statement that something expected is absent.

(ii) **Authority follows verifiability.** Whether S may raise a finding automatically, raise it subject to a check, or only flag it for a human is a function of how deterministically the element of K can be checked against d.

(iii) **Growth.** K is updated incrementally from the interaction between expert and system, without a separate annotation process.

(iv) **Bounded growth.** Every update is attributable to a human decision, constrained by (ii), and reversible.

We use these rather than issue-level accuracy as the primary lens because accuracy against a direct-prompting baseline is overtaken by the next model generation, whereas (i)–(iv) are properties of the organisation, not of the model. Requirements (i)–(iv) are not free-floating desiderata: each is realised by a specific mechanism, and each admits an instrument that a deployment or an offline study can provide (Table 2). Naming the instrument matters for reading our results honestly, because the two requirements that concern knowledge in use are the two we do not yet measure.

3.2. Verifiability-Typed Rules

A rule package is a YAML file validated on load. Each rule has an identifier, a review dimension (capacity, security, interface, channel, data synchronisation, configuration, monitoring, documentation), a severity (blocker, major, minor), a *tier*, the question put to the LLM and the expert, a rationale, a default remediation, an optional applicability precon-

Table 2. Design requirements, the mechanism that enforces each in the implementation, and the instrument that measures it. The last column states what this paper reports and what it deliberately leaves open.

Requirement	Mechanism	Instrument	Reported here
(i) Auditability	every finding carries rule id, severity, tier and either a source range or a verifier-produced absence record; the evidence gate rejects anything else	evidence validity (share of model registrations that passed the gate), gate rejection reasons per review	yes: 79.2% validity, 11/53 rejections analysed (Section 3.4)
(ii) Authority follows verifiability	three-valued tier per rule; static load-time invariants; tools refuse a finding whose authority the tier does not grant	per-tier precision and recall; share of blocker rules that are hybrid	yes: 10/13/5 hard/hybrid/soft; all four blockers hybrid (Section 3.2)
(iii) Growth	expert decisions written as typed events; per-rule statistics, overrides, exemplars and drafts provide the next review's context	acceptance rate per review index; rewrite rate; candidate rules promoted	no: requires an online pilot; the design and the storage schema are reported (Section 3.6)
(iv) Bounded growth	tier is immutable to the learned layer; every write is audited; effective version stamped per review; reset is one audited step	audit coverage; share of changes traceable to a decision; degradation and reset simulation	no: instruments specified, not yet run

dition, a verifier specification and a status. The package also carries a lexicon—heading aliases, placeholder tokens, negation tokens and keyword families (e.g. *loadtest* = load, stress and performance testing in Chinese)—which rules reference by key. Rules never embed vocabulary: the lexicon is the only place where language lives, so a package can be translated or extended without touching verifier code.

The tier is the type of the knowledge (Table 3). The criterion is operational: *can a deterministic procedure produce located evidence for or against the requirement, including evidence of absence?* If yes for every aspect, the rule is hard; if a procedure can locate where the requirement would be addressed and flag surface cues (negation, placeholder, unjustified assertion) but the judgement requires reading, it is hybrid; if no procedure can locate relevant text reliably, it is soft. The three types differ not in confidence but in *who is allowed to speak*, which is the point of the type: it is a permission, not a probability.

A tier decides who may speak; it does not decide what is worth reading. Each rule therefore also carries an *output channel*. Findings from opinion rules become numbered clauses of the formal opinion and are decided one by one. Findings from checklist rules—chapter completeness, annex numbering, table columns, sequence-diagram message counts—become a separate annex of correction items that the reviewer accepts or waives in a single action. Both channels are produced by the same verifiers and both count in the verdict policy (Section 3.5); the split separates what blocks a proposal from what merely has to be tidied, so that a reviewer is not asked to adjudicate a dozen facts that are certainly true and rarely decisive.

Two consequences follow. First, *absence is verifier-owned*: "load testing is not mentioned" cannot be quoted, and an LLM claim of it is unverifiable by construction; only hard verifiers emit absence evidence, a hybrid rule with no candidate becomes pending, and an LLM submission of absence—or a quote beginning with "not mentioned", "missing" and similar—is rejected. This is not a stylistic preference. Absence is the most common defect class in our corpus (98 of 253 planted defects), and it is exactly the claim that a

Table 3. Tier semantics. "Who may raise a finding" is enforced in the tools, not in the prompt: a model that tries is refused by code.

Tier	Verifier	Outcomes	Who may raise a finding
hard	required; every sub-check decisive	pass, fail, not applicable	only the verifier; an LLM attempt is rejected
hybrid	required	candidate, pass (structural only), pending, not applicable	structural fail automatic; LLM only inside candidate spans; no candidate → pending, never fail
soft	forbidden	—	LLM may register; always pending, never counted

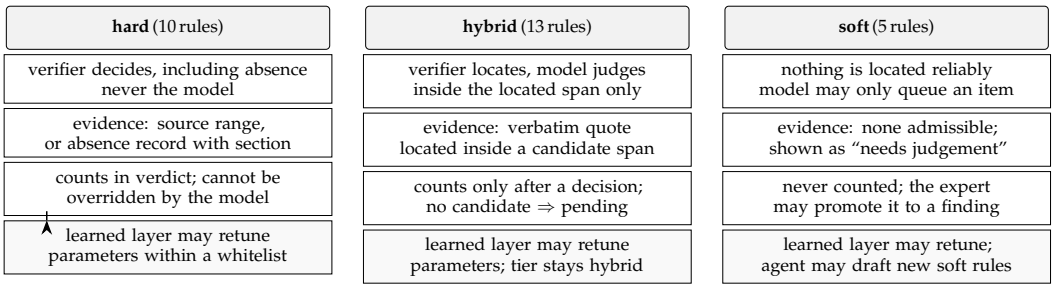


Figure 2. The type is a permission. Reading down a column gives what the tier allows, what evidence it admits, what it contributes to the verdict, and what the learned layer may change. The type itself is not changeable by the learned layer; promotion of a rule from soft to hybrid or hard requires a verifier a human has reviewed.

generative model produces fluently and wrongly; a system that accepts it has converted its most frequent true finding into its most frequent false one. Second, *candidate existence is not satisfaction*: a hybrid statement check never yields pass, and if the LLM registers nothing the checklist says "reviewed, no issue" rather than "pass".

The type is checked statically when the package loads, and a package is rejected as a whole if a rule breaks an invariant: a soft rule with a verifier; a hard or hybrid rule without one; a hard rule whose verifier tree contains a non-decisive check or a pending no-candidate branch; a hybrid rule whose statement check is decisive or whose no-candidate behaviour is fail; a missing lexicon key; or a duplicate identifier. These invariants are the executable form of (ii): a rule whose stated verifiability and whose authority disagree cannot be shipped. They also survive editing, which matters because the package is edited by domain experts: the check runs on load, so a rule that is retyped without a matching verifier fails the next review rather than silently gaining authority.

Table 4 gives the composition of the package after typing the legacy checklist. The distribution is the empirical core of the paper's diagnosis. Only 10 of 28 rules are deterministically verifiable; 13 need a located passage and a reader; 5 cannot be located at all. More importantly, the four *blocker* rules—rate limiting and concurrency control, encryption and signing, a closed authorisation chain, data-exposure assessment—are all hybrid, and all ten hard rules are major or minor. Severity and verifiability are inversely related in this domain, which is the opposite of what a checklist that treats all rules as keyword checks implies: the most consequential requirements are the ones a program can locate but not decide. Seven rules carry applicability preconditions, which turns "not applicable" into an explicit checklist state rather than a silent pass; the lexicon has 25 keyword families, 11 heading groups, 12 placeholder tokens and 11 negation tokens.

Table 4. Composition of the shipped rule package after typing the legacy checklist (28 rules). The four blockers are CAP-001, SEC-001, SEC-002 and SEC-004, all hybrid.

Dimension	Hard	Hybrid	Soft	Total	Blocker	Major	Minor
Capacity	2	3	1	6	1	5	0
Security	0	4	1	5	3	2	0
Interface	2	3	0	5	0	3	2
Documentation	6	0	2	8	0	4	4
Channel	0	1	0	1	0	1	0
Data synchronisation	0	1	0	1	0	1	0
Configuration	0	0	1	1	0	0	1
Monitoring	0	1	0	1	0	0	1
Total	10	13	5	28	4	16	8

3.3. Verifiers

Documents are Markdown; Word and PDF inputs are converted by a pre-existing office plugin of the host, so the review plugin contains no converter and review evidence is always text that the parser saw. The parser builds a heading tree with section paths from both Markdown headings and bold inline headings, tables with cell ranges, Mermaid sequence diagrams [39] with participants and messages, and a normalised text layer (NFKC folding, full-width to half-width punctuation, emphasis removal, whitespace collapsing) with an offset map back to the source, so any match is reported as a source range rather than as a string position in a normalised copy. Proposals in this domain are Chinese with embedded English identifiers, so sentences are split at Chinese and Latin terminators and at table-cell boundaries, and a table cell is itself a candidate unit; absence evidence carries a zero-length range at the position where the expected content would appear, together with the section that should have contained it.

Six verifier families and two combinators cover the package (Table 5). The families differ in the kind of evidence they can produce, and that difference, rather than the check itself, is what the type is calibrated against: *section* can say that a heading is missing, which is an absence record; *table* can point at a cell; *statement* can point at a sentence and mark why it is suspicious; *quantity* can point at a number and its unit; *mermaid-sequence* can name the participants and messages that are missing; *cross-ref* can point at a reference that does not resolve. A rule is hard only when every aspect of its requirement is covered by a family that can decide without reading; it is hybrid when the family can localise but not decide.

Three design choices inside the families are worth stating, because each addresses a failure observed in the prototype. Negation is detected within a window around the matched family rather than in the sentence alone, so “the plan does not include a load test” and “load test: not planned” are both caught. Placeholder detection is lexical and structural: the placeholder tokens from the lexicon, but also a body whose entire content is punctuation, an image, or a phrase such as “to be determined”. And a Mermaid diagram is required by the two diagram rules while an embedded image is a fail, not a substitute: an image cannot be checked for participants and messages, so accepting it would be an unaudited pass. All three choices move work from the model to the verifier, and each costs recall on prose that is well-formed but unusually phrased—a trade-off we quantify in Section 4.3.

3.4. The Evidence Gate

The LLM writes findings only through one tool whose implementation runs the gate (Algorithm 1). A finding is confirmed only if every step passes; otherwise it is stored as pending with the full list of reasons, so the agent can repair the quote once and the reasons can be analysed later. Gate decisions are deterministic and recorded per review, which is what makes “evidence validity” a measurable quantity rather than a claim.

Table 5. Verifier families. "Uses" counts occurrences in verifier trees (a rule may nest several); "Dec." marks families that a hard rule may depend on exclusively.

Family	Uses	Dec.	What it establishes	Evidence emitted
statement	16	yes	sentences hitting a keyword family, negation within a window, assertions without causal justification	sentence range, cue class, candidate span for the model
section	9	yes	heading present, body over a minimum length, body not a placeholder	body range; absence record with section path
table	3	yes	required column groups, placeholder cells, minimum rows	table and cell ranges
mermaid-sequence	3	yes	Mermaid present (an image is a fail), participant and message counts, required message families	element ranges; absence record for missing participants or messages
cross-ref	3	yes	every "see Attachment n" resolves to an entry in the attachment list	range of the reference, or absence record
quantity	1	yes	a number with a unit near a context family	range of the quantified sentence
all, first	4, 2	comb.	conjunction; ordered fallback between locating strategies	union of child evidence

Algorithm 1: Evidence gate (simplified). Rejections are recorded, not raised as errors: a refused finding becomes a pending item that the expert still sees.

Input: review, rule id (optional), evidence list
Output: confirmed finding, or pending item with reasons

resolve rule; if undefined **then** reason ← no rule, tier ← soft
if tier = soft **then** reason ← soft rule
end if
if tier = hard **then raise** hard-rule-verifier-owned
end if
if evidence empty **then** reason ← no evidence
end if
for all e in evidence **do**
    if kind(e) = absence **or** quote starts with an absence marker **then** reason ← absence not allowed
    end if
    $q \leftarrow$ normalise$(e.\text{quote})$; **if** $|q| < 8$ or $|q| > 300$ **then** reason ← bad length
    $h \leftarrow$ locate(q, d) preferring the stated section, then candidate spans, then first hit
    if h = none **then** reason ← quote not found
    else e.range ← h
    end if
    if tier = hybrid **and** h outer to candidates **then** reason ← out of candidates
    end if
end for
return confirmed **if** no reason, pending otherwise

Two properties of the gate deserve emphasis, because they are what distinguishes it from a prompt instruction to "quote the document". First, the gate is *positional*: it checks that the quotation lies inside the span the verifier chose for that rule, not merely that it occurs somewhere in the document. Without that check a model can satisfy a traceability requirement by quoting a true but irrelevant sentence, which is the failure mode reported for citation-generating models where quotes exist for most claims yet substantiate far fewer [18]. Second, the gate is *authority-aware*: it refuses findings on hard rules and on soft rules before looking at the evidence, so a model that ignores the type cannot gain authority by quoting well. The cost of the design is visible in the algorithm—a rejected finding is

downgraded rather than deleted, so the expert still sees the item and the gate's own error rate can be audited.

3.5. Deterministic Verdict Policy

The suggested verdict is a function of the confirmed findings, never of anything the model writes:

$$\text{confirmed} = \{\, i : \ \text{stat}(i) \in \{\text{conf}, \text{acc}, \text{rw}\} \ \wedge \ \text{tier}(i) \in \{\text{hard}, \text{hyb}, \text{exp}\} \,\},$$
$$m = |\{i \in \text{confirmed} : \ \text{sev}(i) = \text{minor}\}|,$$
$$\text{verdict} = \begin{cases} \text{Fail}, & \text{if } \exists i \in \text{confirmed} : \ \text{sev}(i) = \text{blocker}; \\ \text{Revise}, & \text{if } \exists i : \ \text{sev}(i) = \text{major} \ \text{ or } \ m \geq \theta; \\ \text{Pass}, & \text{otherwise.} \end{cases}$$

Soft and pending items are listed but never counted; checklist-channel findings are counted like any other confirmed finding but are rendered outside the numbered clauses, as correction items. Before review only verifier- and gate-confirmed findings count; after review dismissed findings drop out, accepted and rewritten ones stay, and expert-added findings (tier *expert*) count. The policy is recomputed after every decision and shown as the recommendation when the final verdict is requested. The final verdict is written by a single tool action that requires a named reviewer, so no other code path can write it, and the difference between recommended and final verdict is stored as a measure of rubber-stamping.

The policy has one free parameter, the minor threshold θ (three in the shipped configuration). It is exposed rather than learned, and we report its sensitivity in Section 4.5: the alternative—letting the model decide how many minor findings are enough—would return the verdict to the component whose reliability the design is meant to bound. The asymmetry between blocker, major and minor is not a modelling convenience: it encodes the organisation's rule that any blocker is disqualifying, and because all four blockers are hybrid, that rule can only fire after the expert has confirmed the blocker. The policy therefore cannot over-call on an unconfirmed blocker, and Section 4 shows that it also does not over-call in practice.

3.6. Review as Feedback: Bounded Evolution

The review happens in one conversation with a preset of the agent harness—a persona, a skill document and a tool allow-list—that prescribes three phases. In *preliminary review* the expert pastes Markdown, gives a path or attaches a Word file; the agent parses it (resuming a re-reviewed proposal by content hash), runs the checks, and for each hybrid candidate first recalls the rule's acceptance statistics, recent dismissal reasons and phrasing exemplars, then judges the candidate snippets only and registers findings with a verbatim quote. The gate decides. The status view renders the suggested verdict with counts, findings ordered by severity with quotation and section, the items needing judgement, and a one-line checklist summary.

In *expert review* the agent presents findings in batches of at most five through the harness's structured question tool, each with the quotation and the proposed remediation and with three options: fix as suggested, no change needed, fix but in my words. Free text is normalised by the tool rather than by the model: "agreed" becomes accept; "not an issue" or "already covered" becomes dismiss with the remainder stored as the reason; "change it to:" becomes rewrite; "make it minor" becomes accept with a per-finding severity override that does not touch the rule; "you missed" becomes add, matched to the closest rule if any. "Accept the rest" accepts only confirmed findings, and pending items must each be looked at. In *finalisation* the system renders the formal opinion, delivers it as a file and atomically updates the knowledge layer; a later decision reopens the record and requires

a new version. The protocol is machine teaching by ordinary work [28]: the expert never labels anything, and every decision the expert must take anyway becomes a typed event.

Knowledge lives in two layers. The *shipped* layer is the rule package; the *learned* layer is a set of records merged over it at read time (Table 6). Every write appends an audit entry, and an *effective knowledge version* `pack@ver+ovN` is stamped on every review, where N counts rules with an active override, so a review can be reproduced against the knowledge that produced it.

Table 6. The learned layer: seven tables in one storage domain, with the write path and read purpose of each. No table is written by the model alone; the agent proposes, the closing of a review or an explicit expert action commits.

Table	Records	Written by	Read by
`rule_overrides`	override of status, severity, suggestion or a whitelisted verifier parameter; audit entries; draft rules	expert decision via `rules_propose`; parameter change through the review flow	engine on load; rules page
`reviews`	one review: digest, suggested verdict, final verdict, timings, effective version	finalisation	dashboard; evaluation
`issues`	each finding with tier, severity, evidence and gate outcome	`review_check` (hard), `review_record_issue` (hybrid), decide (expert)	finalisation; evaluation
`feedback`	per finding: accept, dismiss, rewrite with reason and replacement text	`review_decide`	memory aggregation
`exemplars`	per rule, phrasing exemplars from rewrites and accepted suggestions	aggregation at finalisation	agent when drafting a suggestion
`rule_candidates`	expert-added findings that matched no rule, plus the draft rule the agent writes for them	finalisation; `rules_propose`	rules page; expert approval
`eval_runs`	offline evaluation runs and per-rule results	evaluation script	dashboard; this paper

The boundary is deliberately asymmetric. The system may change parameters within the whitelist, phrasing, severity and status, and may draft new *soft* rules, but it can never raise a rule's tier: a draft claiming to be hybrid or hard must carry a verifier a human has reviewed. Promotion of authority is thus a human act by construction, which is our reading of (iv). Reversal is symmetric: setting a field back to its shipped value deletes the override, so "changed back" and "never changed" are indistinguishable in the effective version, and a reset removes all overrides of a rule in one audited step while keeping its statistics.

3.7. Implementation

We implemented the design as *HARK* (Human-Gated Authority-Typed Review Knowledge) [40], a plugin bundle for an open-source conversational agent harness that already provides the conversation loop, model access, file access, a structured question tool, a delivery tool, a persona/skill mechanism, a JSON-backed storage service, a web server with access control and layout slots. The plugin adds a pure-function engine (parser, verifiers, gate, policy, memory aggregation, opinion renderer; no I/O and no clock), a storage domain with the seven tables of Table 6, nine tools exposed only inside the review preset (Table 7), three slash commands that query state without the model, an HTTP API under the host's access control, and a dashboard with an overview, a rules page and a sidebar assistant. There is no separate backend, database, embedding index or front-end build; the client is a single 84 kB bundle.

Table 7. The tool surface of the review preset. Authority is enforced by the tool, not by the prompt; the last column lists the refusals that the tool raises as typed errors instead of silently succeeding.

Tool	Purpose	Refusal conditions (selection)
`review_parse`	Markdown to structured document with offsets; resumes by content hash	empty document; unsupported input
`review_check`	runs the verifier tree; registers hard findings and hybrid candidates	unknown rule; package fails load-time invariants
`review_record_issue`	the model's only write path for a finding; runs the gate	hard or soft tier; absence evidence; quote not located; quote outside candidates
`review_status`	suggested verdict, grouped findings, pending items, checklist state	unknown review
`review_decide`	the expert's decision on one finding, or a final verdict	decision after finalisation; final verdict without a named reviewer; verdict not recomputed
`review_finalize`	renders the opinion, delivers the file, closes the review, aggregates memory	open pending items; unconfirmed final verdict
`rules_list`, `rules_propose`, `memory_recall`	inspect the effective package; draft a soft rule; read per-rule statistics and exemplars	tier promotion; unknown lexicon key; duplicate id

Tool contracts are schema-checked with thirteen typed error codes, re-running a check after a decision is refused (it would orphan feedback), and an already reviewed proposal is resumed rather than duplicated. The engine's purity is what makes the offline evaluation of Section 4 possible: the same functions that answer a live review are called by the evaluation script over a corpus, with the model replaced by a deterministic judge, so the mechanism is measured without model variance. On the reference machine the full corpus of 100 proposals $\times$ 23 verifiable rules is processed in about 1.6 s, well under a millisecond per rule–document pair, which is negligible next to the model latency of a single hybrid judgement; the review cost is dominated by the number of candidate spans that reach the model, not by the deterministic layer.

The plugin is installed, like any other, as a profile plugin of the harness; the rule package is data, so a second domain is a second YAML file plus a lexicon, and rule edits take effect on the next review without a restart. The dashboard is served by the host's own web server under the host's access control, so no additional authentication surface is introduced; the left panel shows the review overview (calendar, trend, proposal list with in-place decision), the rule state (shipped, learned, candidates) and the evaluation runs, and a right sidebar tab keeps the active review beside the conversation.

4. Results

4.1. Setup

Corpus.

The released generator (seed 20261029) produced 100 proposals with 253 planted defects over the 23 verifiable rules—98 *absent*, 37 *negated*, 52 *placeholder* and 66 *keyword-present-but-empty*—from phrase pools in which each rule has at least three satisfied and four unsatisfied phrasings, including at least one that avoids the lexicon keywords. Expected verdicts are 30 Pass, 47 Revise and 23 Fail, with 30 proposals shaped on the 15/14/1 verdict distribution of 30 historical expert opinions. The corpus is synthetic because the original proposals could not be released; it measures mechanism correctness and relative baselines, not business-semantic judgement. The released self-check recomputes, for every proposal, that each satisfied rule passes its verifier and that each planted rule fails it, so a reported false positive is attributable to the mechanism rather than to the corpus.

The corpus tested the shipped pack.

The self-check also caught a defect in the shipped package rather than in the baseline. The first generator run failed on every proposal that listed a prototype attachment: the annex-list pattern of DOC-002 captured the word after the annex numeral rather than the numeral itself, so the cross-reference resolver compared "prototype" with "1" and reported a correctly listed attachment as absent from the list. The pattern was corrected to capture the numeral, and the case is now part of the released self-check. A defect of this shape is invisible in a single review and unmissable across a hundred, which is the practical argument for shipping the corpus next to the rules.

Configurations and metrics.

We compare B0, which replicates the 2025 prototype (3–5 substrings per rule, no negation detection, no structure, no evidence) but applies the same verdict policy more leniently than the prototype, with the typed design S-full (verifiers, candidates, gate, policy, memory) and with S−verifier, where hybrid statements are evaluated over the whole document. The plugin ships no model client; hybrid judgements go through a pluggable judge. We report the deterministic *flag judge*, which raises a finding on a candidate span exactly when the verifier marked it negated, assertion-only or placeholder. It measures the mechanism, not LLM capability; the LLM-judge configurations (direct prompting, gate disabled, model-stated verdict) are left to a companion report rather than approximated. Alignment is at (review, rule) granularity, and we separate two quantities that are often conflated: *in-pack agreement*, a predicted rule that matches a planted defect or a satisfied requirement, and *out-of-pack additions*, expert findings for which no rule exists. Only the first is comparable with the baseline; the second is reported separately as candidate rules (Section 3.6) and never folded into precision or κ. We report precision, recall, F1 with bootstrap intervals over proposals, severity-weighted recall (weights 3/2/1), verdict accuracy, macro-F1, Cohen's κ unweighted and quadratically weighted [22,23], evidence validity (share of model registrations that passed the gate) and pending coverage (share of misses shown to the expert as open items). Three repetitions of each configuration produced byte-identical metrics, so the intervals are bootstrap intervals over proposals, not repetition variance.

Table 8. Issue-level and verdict-level results on the synthetic corpus (100 proposals, 253 planted defects; deterministic configurations; 95% bootstrap CIs by proposal; Acc. = verdict accuracy, mF1 = macro-F1; S-full and S−verifier are identical under the deterministic judge).

Configuration	P	R	F1 [95% CI]	R_w	Acc.	mF1	κ	κ_w
B0 keyword	0.275	0.186	0.222 [0.176, 0.267]	0.162	28.0%	0.275	−0.043	−0.059
S-full	**1.000**	**0.664**	**0.798** [0.762, 0.836]	**0.649**	**85.0%**	**0.832**	**0.763**	**0.739**
S−verifier	1.000	0.664	0.798 [0.762, 0.836]	0.649	85.0%	0.832	0.763	0.739

4.2. Typing versus Keywording (RQ2)

B0 produced 47 true, 124 false and 206 missed findings; S-full produced 168 true findings, no false finding and 85 misses, a gain of $\Delta F1 = +0.564$ (paired Wilcoxon signed-rank over per-proposal F1, $p < 0.001$; McNemar on verdict correctness $b/c = 64/7$, $p < 0.001$; both survive Holm correction). S-full reaches F1 = 1.0 on 42 of 100 proposals and its per-proposal F1 has median 0.857.

B0's errors have the shape the typing predicts, and the shape is quantitative. On the 13 rules whose keyword is uninformative (negated, placeholder or hollow phrasings dominate), B0's precision is exactly 0.00; all 124 of its false positives come from six rules, three of which key on vocabulary that satisfied text uses just as often (encryption, channel

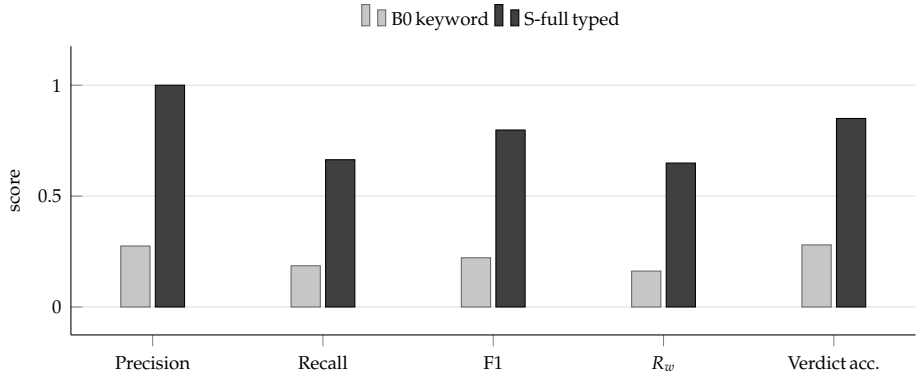


Figure 3. Issue-level and verdict-level scores. Precision is saturated by construction—the gate admits no unlocated quote—so the informative quantities are recall and its weighted form, which carry the cost of admitting the model only inside verifier-chosen spans.

identifiers, rate limiting). Recall is no better: below 0.15 on 15 of 23 rules, including the sequence-diagram rule (0.13) and the load-test rule (0.29), because the satisfied phrasings often avoid the lexicon—the capacity drill that never says "load test". Its verdict agreement is at chance ($\kappa = -0.04$; 17 of 30 Pass proposals rated Fail), reproducing the 2025 failure in which a declared-but-unverifiable checklist drove the verdict.

Table 9 breaks the same findings down by tier and severity. The tier result is the mechanism claim of the paper: hard rules are decided with precision 1.000 and recall 0.880, hybrid rules with precision 1.000 and recall 0.387; under B0 the corresponding precisions are 0.698 and 0.085. Typing thus does not merely improve the aggregate; it removes the failure mode that produced false findings, and what remains is a recall cost that is concentrated where judgement is genuinely needed.

Table 9. Results by tier and by severity (S-full vs. B0). Recall differences measure what each authority path can decide; the severity rows show that the most consequential rules have the lowest recall because all four blockers are hybrid.

	S-full typed				B0 keyword			
Group	TP	FP	FN	P / R	TP	FP	FN	P / R
hard tier (10 rules)	125	0	17	1.000 / 0.880	37	16	105	0.698 / 0.261
hybrid tier (13 rules)	43	0	68	1.000 / 0.387	10	108	101	0.085 / 0.090
blocker (4 rules)	17	0	18	1.000 / 0.486	3	56	32	0.051 / 0.086
major (16 rules)	99	0	42	1.000 / 0.702	22	53	119	0.293 / 0.156
minor (8 rules)	52	0	25	1.000 / 0.675	22	15	55	0.595 / 0.286
all (23 rules)	168	0	85	1.000 / 0.664	47	124	206	0.275 / 0.186

The severity rows carry a warning for the organisation. Best recall is not on the blockers but on the majors (0.702 against 0.486), because the four blocker rules are precisely the rules whose satisfaction cannot be read off the surface: an authorisation chain that is closed, an encryption scheme that is sound and a rate-limiting policy that is adequate all require a reader. A checklist deployed as keyword checks therefore fails most often on the findings that block a release, which is the empirical form of the verifiability tension.

4.3. Where the Remaining Misses Are (RQ1)

All 85 misses are either planned pending states or a documented limit of decisive verifiers; none is a silent pass on a located defect. Sixty-eight are hybrid defects reported as *pending*—27 absent and 30 hollow phrasings the flag judge cannot decide, plus 11 negated or placeholder spans rejected by the gate—so 80% of all misses are shown to the expert as open items rather than hidden; a review surfaces 6–13 pending items, median 9. The

remaining 17 are hard rules decisive on keyword presence (load-test statement 7, channel identifier 5, change-list table 5) that miss placeholder or hollow phrasings, the documented trade-off for making these rules fully automatic.

Table 10 and Figure 4 separate the mechanism's reach by defect kind, and the contrast with the keyword baseline is the clearest single statement of what typing buys. The keyword baseline finds absence only when the keyword is missing entirely (45 of 98) and essentially nothing of the three other kinds (1 of 37 negated, 0 of 52 placeholder, 1 of 66 hollow)—its apparent capability on absence is also the source of its false positives, because it cannot distinguish "the keyword is missing because the requirement is unmet" from "the keyword is missing because the author used another word". The typed design moves all four kinds upward, and the ordering that remains—absent 0.72, negated 0.84, placeholder 0.81, hollow 0.36—is the ordering of what structure alone can decide.

Table 10. Detection by planted defect kind. A defect is *found* if a confirmed finding names its rule, *pending* if the item reaches the expert without being counted, and *silent* if it is not surfaced at all.

Planted kind	Found	Pending	Silent	Recall (found / planted)
Absent (98)	71	27	0	0.72
Negated (37)	31	6	0	0.84
Placeholder (52)	42	5	5	0.81
Hollow (66)	24	30	12	0.36
Total (253)	168	68	17	0.66

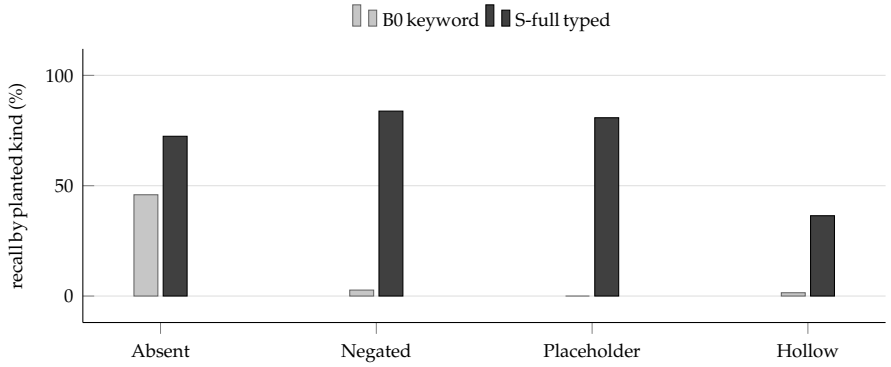


Figure 4. Recall by planted defect kind. The keyword baseline has one capability—noticing that a word is missing—and no way to distinguish that from a paraphrase; the typed design is strongest exactly where keyword logic is blindest, and weakest on hollow phrasings, which require the model and therefore remain pending in the deterministic configuration.

4.4. Evidence Gate

The deterministic judge submitted 53 hybrid findings; 42 passed and 11 (20.8%, in 10 proposals) were rejected as too short, because negated or placeholder sentences fall below the eight-character quote floor. The rejected findings were downgraded to pending rather than discarded—they are the negated and placeholder spans the judge could not quote—so the gate cost 11 located true findings (recall −0.043) and no false one, and each stays visible to the expert. No absence, not-found or out-of-candidates rejection occurred, as expected for a judge that quotes the verifier's own span. Evidence validity is therefore 79.2%: one registration in five fails the first attempt, and the failure is always a property of the evidence supplied, never of the finding's underlying truth.

That number should be read as an upper bound on the gate's usefulness with a real model, not a lower one. The deterministic judge already quotes the verifier's span and still trips the length floor; a generative model quoting from memory will also trip the location

test, and the gate's purpose is to make that failure visible and repairable instead of silent. The result also identifies the quote floor, chosen to stop one- or two-word "evidence", as a parameter to revisit for whole-sentence placeholder spans.

4.5. Span Scoping and Verdict Policy

S−verifier is identical to S-full under the deterministic judge (ΔF1 $= 0$, $b/c = 0/0$): widening the search region added candidate spans but none carried a cue, and the judge never cites outside a region, so the contribution of span scoping is a property of an LLM judge and is measured there. S-full's suggested verdict agreed with the planted verdict in 85 of 100 proposals ($\kappa = 0.76$; confusion over Pass/Revise/Fail $[[30, 0, 0], [6, 41, 0], [4, 5, 14]]$). All 15 disagreements are under-calls caused by pending hybrid findings—nine Fail proposals whose blocker defect was hybrid, six Revise proposals whose major defects were hybrid—and there is no over-call, consistent with a policy that counts only confirmed findings.

The one free parameter behaves the way a policy parameter should (Table 11): at $\theta = 1$ a single minor finding already forces Revise, which over-calls on 15 Pass proposals; from $\theta = 3$ onward the recommendation is unchanged, because in this corpus proposals with three or more minor findings also carry a major finding. We keep $\theta = 3$ and note the plateau as evidence that the verdict is not sensitive to the exact value within a plausible range—a property that a model-written verdict would not have.

Table 11. Sensitivity of the suggested verdict to the minor threshold θ (S-full, 100 proposals). Accuracy and Cohen's κ against the planted verdict.

θ	1	2	3 (shipped)	4	5
Verdict accuracy	72.0%	82.0%	85.0%	85.0%	85.0%
Cohen's κ	0.534	0.713	0.763	0.763	0.763

4.6. One Proposal, End to End

Table 12 follows a single proposal (`syn-hist-30`, a balance-query capability opened to third-party channels) through the four authority paths. Quotations are translated from the original Chinese; the untranslated text is in the released corpus. The case is chosen because it exercises every path at once: a hard fail the model never saw, three confirmed hybrid findings, and one pending item that keeps a miss visible.

The walkthrough also shows what the design does *not* claim. The tracking defect is missed by the mechanism—the phrase plans tracking but does not implement it, and no verifier could decide that—so it is handed over as an open item. A reviewer who reads the pending list sees five findings, not four, and the difference between the four that count and the one that does not is explicit in the interface rather than implicit in a model's confidence.

4.7. The Protocol Closed on Internal Proposals

Beyond the corpus we ran the system on three technical solution proposals written for the platform, one reviewed twice after revision and once re-run to exercise resumption: five runs in total. The runs were performed by the team that built the system, so they document mechanics, not an independent expert study. Across the five runs the system raised 58 findings: 40 located by verifiers, 12 judged by the model inside candidate spans, and 6 added by the operator. The gate refused 3 of the 12 model-judged findings, all with the same reason—the quote fell outside the spans the verifier had located—so the refusal path of Section 3.4 was exercised on real prose and not only on planted defects. Hard and typed rules rated every proposal *fail*; the two reviews carried to a final verdict kept it, the operator accepting 15 of the 18 findings that required a decision and rejecting 3, none rewritten. The

Table 12. Authority paths on one proposal. The expected verdict is Fail; the system suggests Fail from three confirmed blockers, and the single missed defect is the pending item.

Requirement	What the document contains (translated)	Tier, severity	Path and outcome
Rate limiting / concurrency control (CAP-001)	"Concurrency control plan: /"	hybrid, blocker	placeholder cue in a candidate span; the model quotes the line; gate: located, length ok → *confirmed*
Closed authorisation chain (SEC-002)	"The binding relation is not verified; the user signs in by mobile number directly."	hybrid, blocker	negation cue; quote located in span → *confirmed*
Data-exposure risk (SEC-004)	"Leakage risk assessment: /"	hybrid, blocker	placeholder cue → *confirmed*
Authorisation sequence diagram (DOC-001)	"The authorisation sequence diagram is shown in the figure, omitted." plus a PNG	hard, major	Mermaid required, an image is a fail → *registered by the verifier alone*; the model was never asked
Usage tracking implemented (MON-001)	"New page tracking points must be added to count usage."	hybrid, minor	no candidate span for an implementation statement → *pending*, shown to the expert, never counted

Suggested verdict: Fail (three confirmed blockers); agreement with the planted verdict; per-proposal F1 0.889.

learned layer was written: after five runs it held statistics for 13 rules and 21 exemplars, and no rule had changed tier.

5. Discussion

5.1. Answers to the Research Questions

RQ1: the verifiability tension.

In this corpus the tension is not hypothetical but measurable, and it has a direction: the rules that decide a release are the rules least decidable by structure. All four blocker rules are hybrid; recall on blockers (0.486) is below recall on majors (0.702) and minors (0.675). A checklist that is tuned to be fully automatic will therefore be automated and lenient exactly where leniency is expensive, while a checklist tuned to be strict about blockers will produce machine findings that a human must adjudicate anyway. The three-way typing is the smallest mechanism we found that makes this tension explicit instead of average it away: it assigns each rule an authority path first, and only then measures the path.

RQ2: what typing buys.

Typing removed the failure mode rather than merely reducing it. Under B0 the association between rule declarations and reported findings is broken in both directions: 124 false findings and 206 misses, verdicts at chance. Under S-full, no finding lacks a located quote, and recall rises to 0.664 with severity weighting at 0.649 and verdict agreement at $\kappa = 0.76$. The interesting part is not the F1 gap but its composition: the gain is concentrated on negated (2.7% → 83.8%) and placeholder phrasings (0% → 80.8%), that is, on text that contains the keyword and means the opposite of it. Keyword logic fails there by construction; structure alone recovers most of it, and the model is needed for the rest.

RQ3: whether evidence changes behaviour.

The gate did not make the mechanism more accurate; it made it auditable, and the audit is what changed our own behaviour. One registration in five failed the gate, and the failure was always a property of the evidence rather than of the finding: too short a quote,

never a mislocated one, because the deterministic judge quotes the span it was given. We therefore do not claim that the gate improves precision in this configuration—precision was already saturated—but that it produces a signal a reviewer can act on, and that it makes the weakest link in the chain (a model that quotes from memory) visible as a rejection rather than as an unverifiable claim. Whether it improves a real model's precision is the first question of the planned LLM-judge ablation.

RQ4: whether expert decisions can be the knowledge.

The mechanism stores decisions, not preferences: each review decision is written with an issue identifier, a verdict, an optional rewrite and a timestamp, and the aggregate recall per rule is a query over those rows, not a retrained parameter. That design has a cost we did not pay down—no expert reviewed the 100 synthetic proposals, so the memory layer is exercised only by the deterministic judge—and a benefit we can state precisely: the memory cannot drift from the rules, because it is keyed by the same rule identifiers, and an expert can see and correct any single row. Where the learning lives is therefore a design choice with an audit trail, not an emergent property of a model.

5.2. Why Asymmetric Authority

The arithmetic of the two directions is not symmetric. A false finding costs reviewer trust and, if it reaches the formal letter, an argument with the submitting team; enough of them end the deployment. A missed finding costs one line in the reviewer's own memory—provided the reviewer still has the habit of reading the document, which the pending list is designed to keep. The policy encodes that asymmetry: the system may conclude *Fail* only from confirmed blockers, but it can always be overruled, and it never writes the final verdict. This is also why we do not use the model's own verdict anywhere in the policy: a model-stated verdict is a single point of failure with no audit trail, whereas the policy's inputs are rows a human can inspect one by one.

The asymmetry has a limit we should state. It assumes the reviewer reads the pending list. In our walkthrough the missed tracking defect is one item in a list of five, and a reviewer who trusts the four confirmed findings will sign a Fail for the right reason and still never see the fifth. The interface, not the policy, carries that risk; the plugin addresses it by making pending items first-class objects in the review store and by requiring the reviewer's confirmation to be recorded per item, but a field study is needed to know whether that is enough.

5.3. Position Relative to Automated Reviewers

Recent work has largely measured automated review as a text-generation task: given a diff or a design, produce comments that a developer accepts [26,33]. That framing optimises for agreeableness, and the largest industrial deployment we know of reports acceptance by developers on individual review comments, a metric that suits a diff and a comment stream rather than a verdict. Our framing is the complement: the artifact is not a comment stream but a verdict with an auditable derivation, and the unit of correctness is a rule, which makes the system comparable with a compliance checklist rather than with a human reviewer. The two framings meet where the checklist itself is written in prose: neuro-symbolic verification of design documents [27] shares our impulse to constrain the model with a structure, but uses the structure to check a model's extraction; we use it to decide who is allowed to speak.

The compliance context also sets a constraint we have not fully exploited. Security-requirement reviews and certification audits are already organised around control catalogues [35,36], and a traced requirement is the artifact those processes treat as evidence [37]. A rule pack whose identifiers match the catalogue would let a review quote a control

number directly, and would let the memory aggregate at the level a certification body cares about. We keep the rule schema aligned with that possibility (each rule carries a dimension and a provenance field) but leave the mapping to future work, because it changes the ontology of a review, not its implementation.

Two risks of this framing deserve naming. The first is regulatory: if a review supports a decision that a regulation covers, the artefact and its provenance may acquire legal weight, which is why the AI Act's documentation duties [38] are a plausible next constraint on the design. The second is the model itself: a judge that reads untrusted proposals is a prompt-injection target [32], and a memory that accumulates expert rewrites is a privacy target [31]. Neither is solved by the gate; both are reasons to keep the model's authority narrow, which is the design's main defence.

6. Limitations and Future Work

The corpus is synthetic.

The 100 proposals are generated from phrase pools, so they measure mechanism correctness against planted defects, not judgement quality against real design intent. Three consequences are unavoidable: a satisfied phrasing is satisfied by construction, the vocabulary is narrower than field text, and the verdict labels are correct by construction rather than by expert agreement. We mitigate the third by planting defects at the clause level and deriving the expected verdict from the policy itself (a proposal is Fail iff a blocker is planted), so accuracy measures the policy's execution rather than a taste. The mitigation does not remove the gap; a corpus of anonymised real proposals with two independent expert labels is the natural next step, and the anonymisation procedure is already specified in the released pipeline.

The judge is deterministic.

Hybrid recall (0.387) is a lower bound, since the flag judge decides only on structural cues and leaves semantic cases pending by design; conversely precision (1.000) is an upper bound, because the same judge never invents a quote. Span scoping showed no effect precisely because the judge only cites the span it is given, so the effect that spans are meant to have—keeping a generative model from wandering—is untested here. The planned ablation covers three judge settings (direct prompting over the whole document, prompting over candidate spans, and prompting with the gate disabled) with two open models, fixed decoding parameters and the same corpus, and reports the gate's rejection taxonomy as the primary outcome.

No independent expert was in the loop.

The online form of the contribution—that review decisions become memory, and that memory improves later reviews—is not tested. The five internal runs of Section 4.7 show that the loop closes and that the learned layer is written, but the operator was the team that wrote the rules, so their acceptance rate says nothing about an outside expert, and five runs cannot measure a learning curve. What the runs exercise is the plumbing (decision recording, memory aggregation, exemplar retrieval) and the offline aggregator, not the claim that expert recall per rule rises with exposure. That claim needs a longitudinal deployment: the same reviewers, the same capability, a baseline period without memory and a treatment period with it, with per-rule recall, dismissal rate and rewrite rate as the outcomes. We describe the measurement plan and the schema rather than report the effect, and we consider this the paper's largest open question.

Scope of the rule pack.

Twenty-eight rules covering one capability's review checklist are not a review standard. The typing mechanism is agnostic to the rules—it takes any rule with a declared verifier—but the composition table (Section 3.2) shows that the balance of tiers is a property of the domain: documentation rules are mostly mechanical, security rules are mostly semantic. A different domain will need a different balance, and the question of how to choose tiers without taste is open; our practice was to write the verifier first and the tier second, which makes the choice traceable but not automatic.

Language and lexicon.

The corpus, the lexicon and the rules are in Chinese, for the electricity-service deployment; the mechanism is language-agnostic but the cue lists are not. Porting is a configuration task, not a code change, and the lexicon is versioned with the pack, but each port needs its own self-check, because a cue list calibrated on one domain's phrasing fails silently on another's, and only a per-domain self-check catches that.

Verdict policy and human factors.

The verdict policy is deterministic and its threshold is insensitive in this corpus, but it assumes a reviewer who reads pending items, and we have not measured that. Nor have we measured whether an expert overrides the suggested verdict more often when it is wrong in the lenient direction, which is the asymmetry the design leans on. Both are study designs, not implementation gaps, and both belong with the longitudinal deployment.

7. Conclusions

A review system built on large language models tends to give the model one of two roles: generate the comments, or decide the verdict. We have argued for a third arrangement in which the model never decides alone, the program never decides what it cannot check, and the expert's decisions are the knowledge the system accumulates. Its unit of construction is the rule, its unit of evidence is the quote, its unit of authority is the tier, and its unit of learning is the decision row.

On a corpus of 100 proposals with 253 planted defects, the arrangement produces no false finding, recovers two thirds of the defects, and agrees with the planted verdict on 85% of proposals ($\kappa = 0.76$), against a keyword baseline at chance. The more instructive result is where it loses: the rules that block a release are the rules whose satisfaction needs a reader, and the misses that remain are concentrated on prose that plans rather than implements. The loop also closed outside the corpus: five runs on three internal proposals, 58 findings and 18 operator decisions, with the gate refusing three model-judged findings on real prose. We read all of this as evidence that the design constraint is right—the model's job is to read, not to conclude—and as the agenda that follows: a judge worth trusting, a longitudinal test of whether decisions improve later reviews, and a mapping between rule packs and the control catalogues that compliance work already uses.

Author Contributions: Conceptualization, A.; methodology, A. and B.; software, C.; validation, B. and C.; formal analysis, B.; investigation, D.; data curation, C.; writing—original draft preparation, A.; writing—review and editing, A., D. and E.; visualization, C.; supervision, E. and F.; project administration, D.; funding acquisition, F. All authors have read and agreed to the published version of the manuscript.
Funding: This research received no external funding.
Institutional Review Board Statement: Not applicable.
Informed Consent Statement: Not applicable.

Data Availability Statement: The synthetic corpus, the generator, the judgment traces and the evaluation scripts are available in the artifact repository at https://github.com/yyyyyyzt/HARK-Human-Gated-Authority-Typed-Review-Knowledge (accessed on 4 November 2026) and archived at Zenodo under DOI 10.5281/zenodo.23955205. The 30 historical expert opinions and the three internal proposals of Section 4.7 cannot be released for confidentiality; the anonymisation procedure and the aggregate statistics derived from them are documented in the repository.

Acknowledgments: During the preparation of this manuscript, the author(s) used a large language model assistant for drafting and language polishing. The authors reviewed and edited the content and take full responsibility for the content of the publication.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Huang, J.; Chen, X.; Mishra, S.; Zheng, H.S.; Yu, A.W.; Song, X.; et al. Large Language Models Cannot Self-Correct Reasoning Yet. In Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024), Vienna, Austria, 2024.
2. Liu, N.F.; Lin, K.; Hewitt, J.; Paranjape, A.; Bevilacqua, M.; Petroni, F.; et al. Lost in the Middle: How Language Models Use Long Contexts. *Trans. Assoc. Comput. Linguist.* **2024**, *12*, 157–173. https://doi.org/10.1162/tacl_a_00638
3. Fagan, M.E. Design and Code Inspections to Reduce Errors in Program Development. *IBM Syst. J.* **1976**, *15*, 182–211. https://doi.org/10.1147/sj.153.0182
4. Laitenberger, O.; DeBaud, J.M. An Encompassing Life Cycle Centric Survey of Software Inspection. *J. Syst. Softw.* **2000**, *50*, 5–31. https://doi.org/10.1016/S0164-1212(99)00073-4
5. ISO/IEC/IEEE. *ISO/IEC/IEEE 29148:2018 Systems and Software Engineering—Life Cycle Processes—Requirements Engineering*; International Organization for Standardization: Geneva, Switzerland, 2018.
6. Génova, G.; Fuentes, J.M.; Llorens, J.; Hurtado, O.; Moreno, V. A Framework to Measure and Improve the Quality of Textual Requirements. *Requir. Eng.* **2013**, *18*, 25–41. https://doi.org/10.1007/s00766-011-0134-z
7. Femmer, H.; Méndez Fernández, D.; Wagner, S.; Eder, S. Rapid Quality Assurance with Requirements Smells. *J. Syst. Softw.* **2017**, *123*, 190–213. https://doi.org/10.1016/j.jss.2016.02.047
8. Hemmat, A.; Sharbaf, M.; Kolahdouz-Rahimi, S.; Lano, K.; Yassipour Tehrani, S. Research Directions for Using LLM in Software Requirement Engineering: A Systematic Review. *Front. Comput. Sci.* **2025**, *7*, 1519437. https://doi.org/10.3389/fcomp.2025.1519437
9. Huang, L.; Yu, W.; Ma, W.; Zhong, W.; Feng, Z.; Wang, H.; et al. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *ACM Trans. Inf. Syst.* **2025**, *43*, 42. https://doi.org/10.1145/3703155
10. Dolcetti, G.; Iotti, E. A Dual Perspective Review on Large Language Models and Code Verification. *Front. Comput. Sci.* **2025**, *7*, 1655469. https://doi.org/10.3389/fcomp.2025.1655469
11. Chinnam, H. MARV: A Multi-Agent Approach for Validation of Software Requirements Specifications. Master's Thesis, Tampere University, Tampere, Finland, 2025. Available online: https://urn.fi/URN:NBN:fi:tuni-202507297873 (accessed on 30 October 2026).
12. Alsawalqah, H.; Abadleh, A.; Ibrahim, S.; Alqaralleh, B.A.Y.; Aoun, C.; Zreikat, A.I. Evidence-Aware Human-in-the-Loop LLM Review for Requirements-to-Planning Decisions. *Computers* **2026**, *15*, 608. https://doi.org/10.3390/computers15090608
13. Malik, S.; Singh, A.; Azad, A.P. PolicyGuard: From Organizational Policies to Neuro-Symbolic Compliance Review Engines. *arXiv* **2026**, arXiv:2606.32004.
14. Garcez, A.d'A.; Lamb, L.C. Neurosymbolic AI: The 3rd Wave. *Artif. Intell. Rev.* **2023**, *56*, 12387–12406. https://doi.org/10.1007/s10462-023-10448-w
15. Sun, T.; Xu, J.; Li, Y.; Yan, Z.; Zhang, G.; Xie, L.; et al. BitsAI-CR: Automated Code Review via LLM in Practice. In Proceedings of the Companion Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25), Trondheim, Norway, 2025. https://doi.org/10.1145/3696630.3728552

16. Lin, H.Y.; Liang, M.; Tantithamthavorn, K.; Thongtanunam, P. Is Agentic Code Review Helpful? Mining Developers' Feedback to CodeRabbit Reviews in the Wild. *arXiv* **2026**, arXiv:2607.03316.
17. Gao, T.; Yen, H.; Yu, J.; Chen, D. Enabling Large Language Models to Generate Text with Citations. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023), Singapore, 2023; pp. 6465–6488. https://doi.org/10.18653/v1/2023.emnlp-main.398
18. Zhang, J.; Chen, Y.; Commodore-Mensah, Y.; Oberst, M. Verifiable by Construction: Claim-Level Evaluation of Verbatim Citation in Clinical Question Answering. *arXiv* **2026**, arXiv:2609.15964.
19. Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegreffe, S.; et al. Self-Refine: Iterative Refinement with Self-Feedback. In Proceedings of the Advances in Neural Information Processing Systems 36 (NeurIPS 2023), New Orleans, LA, USA, 2023.
20. Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; Yao, S. Reflexion: Language Agents with Verbal Reinforcement Learning. In Proceedings of the Advances in Neural Information Processing Systems 36 (NeurIPS 2023), New Orleans, LA, USA, 2023.
21. Zhang, J.; Hu, S.; Lu, C.; Lange, R.; Clune, J. Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents. *arXiv* **2025**, arXiv:2505.22954.
22. Cohen, J. A Coefficient of Agreement for Nominal Scales. *Educ. Psychol. Meas.* **1960**, *20*, 37–46. https://doi.org/10.1177/001316446002000104
23. Landis, J.R.; Koch, G.G. The Measurement of Observer Agreement for Categorical Data. *Biometrics* **1977**, *33*, 159–174. https://doi.org/10.2307/2529310
24. Veizaga, A.; Shin, S.Y.; Briand, L.C. Automated Smell Detection and Recommendation in Natural Language Requirements. *IEEE Trans. Softw. Eng.* **2024**, *50*, 695–720. https://doi.org/10.1109/TSE.2024.3361033
25. Wang, T.; Wang, W.; Yang, Y. AirReq: Automated Requirements Smell Detection and Elimination for Commercial Aircraft Systems. In Proceedings of the 2025 IEEE 33rd International Requirements Engineering Conference Workshops (REW), 2025; pp. 173–180. https://doi.org/10.1109/REW66121.2025.00027
26. Pornprasit, C.; Tantithamthavorn, C. Fine-Tuning and Prompt Engineering for Large Language Models-Based Code Review Automation. *Inf. Softw. Technol.* **2024**, *175*, 107523. https://doi.org/10.1016/j.infsof.2024.107523
27. Galitsky, B.; Rybalov, A. Neuro-Symbolic Verification for Preventing LLM Hallucinations in Process Control. *Processes* **2026**, *14*, 322. https://doi.org/10.3390/pr14020322
28. Mosqueira-Rey, E.; Hernández-Pereira, E.; Alonso-Ríos, D.; Bobes-Bascarán, J.; Fernández-Leal, Á. Human-in-the-Loop Machine Learning: A State of the Art. *Artif. Intell. Rev.* **2023**, *56*, 3005–3054. https://doi.org/10.1007/s10462-022-10246-w
29. Chang, Y.; Wang, X.; Wang, J.; Wu, Y.; Yang, L.; Zhu, K.; et al. A Survey on Evaluation of Large Language Models. *ACM Trans. Intell. Syst. Technol.* **2024**, *15*, 1–45. https://doi.org/10.1145/3641289
30. Wang, L.; Ma, C.; Feng, X.; Zhang, Z.; Yang, H.; Zhang, J.; et al. A Survey on Large Language Model Based Autonomous Agents. *Front. Comput. Sci.* **2024**, *18*, 186345. https://doi.org/10.1007/s11704-024-40231-1
31. Wang, B.; He, W.; Zeng, S.; Xiang, Z.; Xing, Y.; Tang, J.; et al. Unveiling Privacy Risks in LLM Agent Memory. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL 2025), Vienna, Austria, 2025; pp. 25241–25260. https://doi.org/10.18653/v1/2025.acl-long.1227
32. Shi, J.; Yuan, Z.; Liu, Y.; Huang, Y.; Zhou, P.; Sun, L.; et al. Optimization-Based Prompt Injection Attack to LLM-as-a-Judge. In Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS 2024), Salt Lake City, UT, USA, 2024; pp. 660–674. https://doi.org/10.1145/3658644.3690291
33. Tantithamthavorn, K.; Zou, Y.; Wong, A.; Gupta, M.; Wang, Z.; Buller, M.; et al. RovoDev Code Reviewer: A Large-Scale Online Evaluation of LLM-Based Code Review Automation at Atlassian. In Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP 2026), 2026; pp. 34–45. https://doi.org/10.1145/3786583.3786851

34. Djouvas, C.; Andreou, C.; Voutsa, M.C.; Tsapatsoulis, N. Model-Contingent Polarity Bias in Large Language Model Annotation: Implications for Semantic Multimedia Personalization. *Computers* **2026**, *15*, 262. https://doi.org/10.3390/computers15050262
35. Zulfigarzada, S.; Gadirli, A.; Karimov, J.; Cerneckas, D.; Rackiene, R.; Azubalis, M. Cybersecurity Requirements and Certification Standards in Industrial Automation Systems: A Systematic Review. *Computers* **2026**, *15*, 364. https://doi.org/10.3390/computers15060364
36. Janisar, A.A.; Meidan, A.; Kalid, K.S.b.; Gilal, A.R.; Sarlan, A.B. Security Requirements Engineering: A Review and Analysis. *Computers* **2025**, *14*, 429. https://doi.org/10.3390/computers14100429
37. Ferreira-Santos, J.; Pombo, L. The Art Nouveau Path: Requirements Engineering and Traceability for City-Scale In-the-Wild Mobile Augmented Reality Learning Services. *Computers* **2026**, *15*, 243. https://doi.org/10.3390/computers15040243
38. European Parliament and Council. Regulation (EU) 2024/1689 of the European Parliament and of the Council Laying Down Harmonised Rules on Artificial Intelligence (Artificial Intelligence Act). *Off. J. Eur. Union* **2024**, L 1689.
39. Mermaid: Generation of Diagrams and Flowcharts from Text. Available online: https://mermaid.js.org (accessed on 30 October 2026).
40. HARK: Human-Gated Authority-Typed Review Knowledge—Artifact Repository for This Paper. Available online: https://github.com/yyyyyyzt/HARK-Human-Gated-Authority-Typed-Review-Knowledge (accessed on 4 November 2026).